

Техническое задание:

1. Стек технологий (обязательные требования)

- **Язык программирования:** Python 3.11 или выше.
- **Фреймворк для парсинга:** На выбор кандидата (Scrapy, BeautifulSoup + Requests, Selenium, Playwright и т.п.) с обязательным обоснованием выбора в комментариях к коду или в README.
- **База данных:** Реляционная СУБД (PostgreSQL, MySQL или SQLite – для простоты тестирования). Допускается использование SQLite в целях упрощения, но приложение должно быть готово к переключению на внешнюю БД через конфигурацию.
- **ORM:** SQLAlchemy (версия 2.0 рекомендуется) для взаимодействия с БД.
- **Контейнеризация:** Docker и Docker Compose для сборки и запуска всего окружения (приложение + БД).
- **Дополнительно:** Использование `asyncio` или `aiohttp` приветствуется, но не обязательно.

2. Функциональные требования

Разрабатываемая программа должна:

1. **Принимать на вход** список идентификаторов (например, ИНН) в количестве от 100 и более. Способ передачи данных:
 - Через аргумент в docker compose из файла (.xlsx).
2. **Для каждого идентификатора** выполнять парсинг целевого сайта согласно алгоритму, описанному в разделе «Задание». Алгоритм может быть представлен в виде текста и/или графической схемы (прилагается отдельно).
3. **Извлекать необходимые данные** (перечень полей будет указан в задании) и приводить их к корректному типу.
4. **Сохранять результаты** в базу данных:
 - Спроектировать схему БД (таблицы, связи) самостоятельно, исходя из структуры извлекаемых данных.
 - Использовать SQLAlchemy для описания моделей и выполнения запросов.
 - Обеспечить уникальность записей (например, избегать дублирования по ИНН и дате парсинга).
5. **Обрабатывать ошибки:**
 - Сетевые сбои (таймауты, недоступность сервера) – реализовать повторные попытки с экспоненциальной задержкой.
 - Отсутствие данных по идентификатору (например, ИНН не найден) – логировать и продолжать работу.

- Блокировки со стороны сайта (CAPTCHA, блокировка по IP) – предусмотреть паузы, смену User-Agent, использование прокси (опционально, можно заглушкой).
6. **Массовый ввод:**
- При обработке 100+ идентификаторов программа не должна «упасть» и должна использовать разумные ограничения по нагрузке (задержки между запросами, конкурентность не более N потоков).
 - Желательно реализовать возможность докачки (resume) при прерывании.
7. **Логирование:**
- Записывать в лог-файл (и/или в stdout) этапы работы, ошибки, предупреждения.
 - Формат логов – информативный (время, уровень, сообщение).

3. Нефункциональные требования

- **Читаемость кода:**
 - Соблюдение PEP 8, осмысленные имена переменных и функций.
 - Комментарии к сложным участкам, документация функций (docstrings).
 - Структурированность проекта (разделение на модули: парсер, модели, работа с БД, конфигурация, утилиты).
- **Документация:**
 - Файл `README.md` с описанием:
 - назначения программы;
 - стека;
 - инструкции по установке и запуску (через Docker и локально);
 - примера использования;
 - описания конфигурации.
- **Контейнеризация:**
 - Наличие `Dockerfile`, который собирает образ приложения.
 - Наличие `docker-compose.yml`, который поднимает сервис парсера и необходимую БД (например, PostgreSQL).
 - Все зависимости должны фиксироваться в `requirements.txt` или `pyproject.toml`.
- **Обработка ошибок:**
 - Программа не должна завершаться аварийно при возникновении исключений, связанных с парсингом отдельных элементов.
 - Должна быть предусмотрена очистка ресурсов (закрытие соединений с БД, сессий).

4. Задание (место для вставки пользователем)

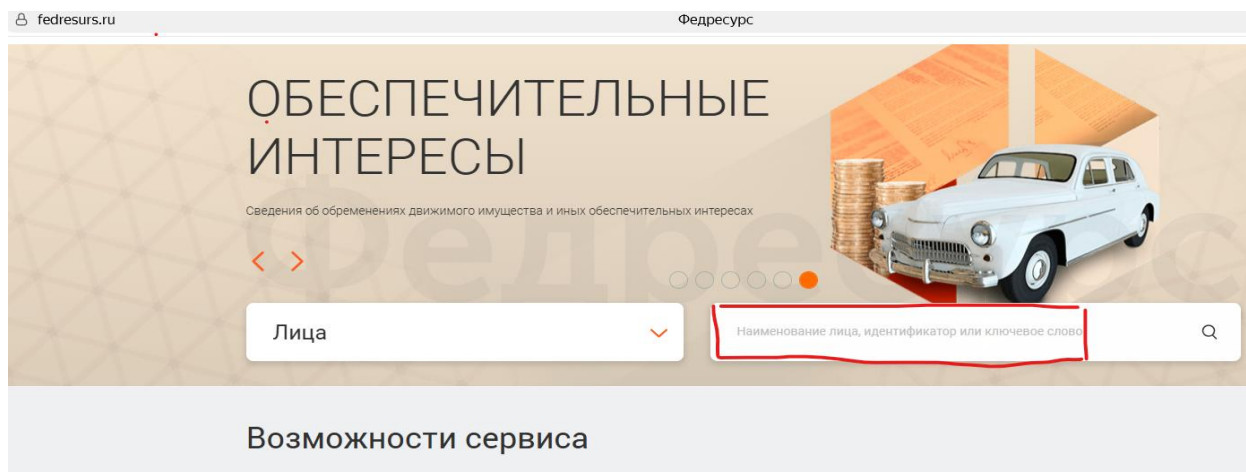
Задача № 1

Цель: Достать данные по банкротству с сайта <https://fedresurs.ru/>

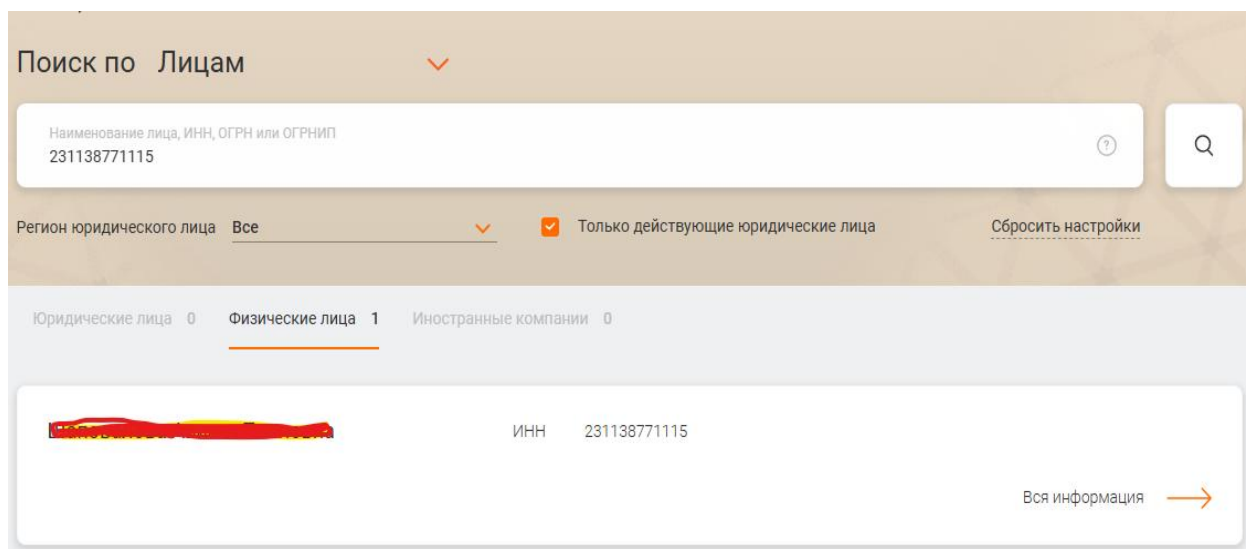
Исходные данные: **ИНН 231138771115**

Алгоритм:

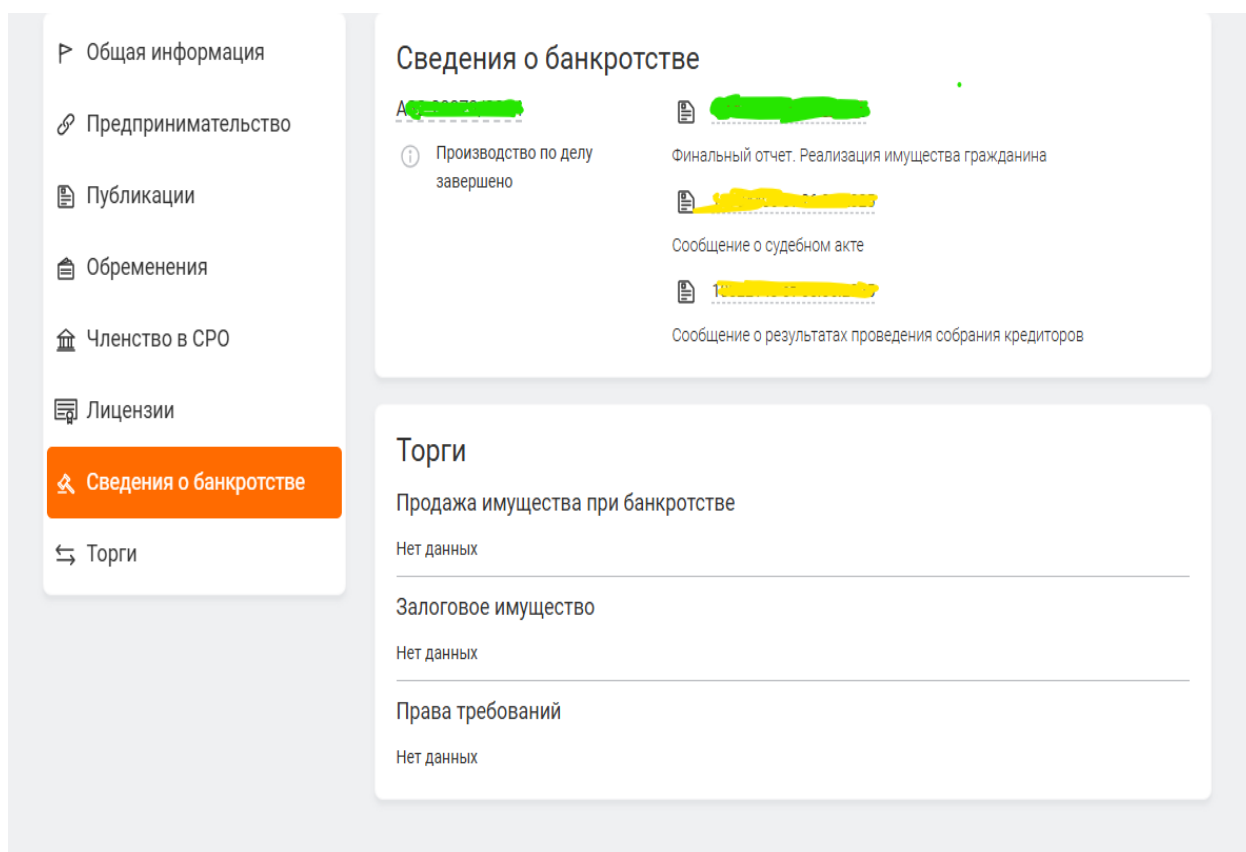
- Вставить ИНН в колонку поиска



- Пройти по стрелочке, где вся информация



- На вкладке Сведения о банкротстве нужно № дела и последняя дата.



Формат полученных данных:

Id	ИНН	№ дела	Последняя дата
1	231 138 771 115	А.....	0..05.2025

Задача № 2

Цель: Достать данные по банкротству с сайта <https://kad.arbitr.ru/>

Исходные данные: **№ дела А32-28873/2024**

Алгоритм:

- Вставить № дела в колонку поиска

kad.arbitr.ru Картотека арбитражных дел

ЭЛЕКТРОННОЕ ПРАВОСУДИЕ

КАРТOTEKA СТРАЖ БАНК РЕШЕНИЙ КАЛЕНДАРЬ ПЕРЕРЫВЫ МОЙ АРБИТР ПРОВЕРКА ЭП О НАС

Фильтр дел Как это работает? А Административные Г Гражданские Б Банкротные Судебные поручения

Участник дела
название, ИНН или ОГРН Любой +

Судья
фамилия судьи +

Суд
название суда +

Номер дела
например, А50-5568/08 +

Дата регистрации дела
с дд.мм.гггг по дд.мм.гггг

Найти
Сбросить все

Укажите атрибуты дела для поиска

- Перейти по № дела

ПРАВОСУДИЕ

КАРТOTEKA СТРАЖ БАНК РЕШЕНИЙ КАЛЕНДАРЬ ПЕРЕРЫВЫ МОЙ АРБИТР ПРОВЕРКА ЭП О НАС

Как это работает? А Административные Г Гражданские Б Банкротные Судебные поручения

Дело	Судья текущая инстанция	Истец	Ответчик
и ОГРН Любой + А32-28873/2024			

Найти

Найдено 1 дел

- На вкладке Электронное дело нужна получить данные: **последняя дата, документ и наименование документа**

The screenshot shows a web browser window displaying a PDF document titled "Определение.pdf". The browser's address bar shows a partially obscured URL. The document is split into two panels. The left panel displays page 1, which features a QR code at the top left and the Russian coat of arms at the top right. Below the coat of arms, there is text identifying the Arbitrazhnyy sud Krasnodarskogo kraia (Arbitration Court of the Krasnodar Krai) and its website. The right panel displays page 2, which begins with the word "ОПРЕДЕЛЕНИЕ" (Decision). The text on page 2 describes the court's proceedings regarding the insolvency of LLC "Krasnodar" and mentions the presence of a court clerk.

Формат полученных данных:

id	№ дела	Последняя дата	Наименование документа
1	A32-28873/2024	0..05.2025	О завершении реализации имущества гражданина и освобождении гражданина от исполнения обязательств

5. Требования к результату

- Готовый код, размещённый в публичном репозитории (GitHub / GitLab / др.).
- Инструкция по запуску в README.md.
- Программа должна успешно запускаться и выполнять парсинг на тестовых данных (кандидат может сам подготовить небольшой набор ИНН для демонстрации).
- В ответе на задание указать ссылку на репозиторий, а также приложить скриншоты/лог работы программы (опционально).

6. Критерии оценки

1. **Полнота реализации функциональных требований** – программа делает то, что требуется.
2. **Качество кода** – структурированность, читаемость, соблюдение стандартов.
3. **Архитектурные решения** – грамотное разделение ответственности, использование ООП, переиспользуемость.
4. **Работа с БД** – корректное проектирование схемы, использование SQLAlchemy (сессии, отношения, индексы).
5. **Обработка ошибок и логирование** – программа не падает на единичных сбоях, информативные логи.
6. **Производительность и масштабируемость** – способность обработать 100+ объектов с учётом ограничений.
7. **Документирование и инструкции** – понятно, как запустить и проверить.
8. **Контейнеризация** – корректная работа через Docker Compose, отсутствие лишних зависимостей.